

PROJECT REPORT

(Project Term January – May 2026)

CodeArena: A Spring Boot Quiz Application with an Automated CI/CD Pipeline using Jenkins and Docker

Submitted by

Pranjal Soni

Registration Number: 12407377

Course Code: PETV137

Under the Guidance of

Dr. Harpreet Kaur

Priyanka Gupta

School of Computer Science and Engineering
Lovely Professional University, Phagwara, Punjab

DECLARATION

I hereby declare that the project work entitled "CodeArena: A Spring Boot Quiz Application with an Automated CI/CD Pipeline using Jenkins and Docker" is an authentic record of my own work carried out as a requirement of the Project for the award of the B.Tech Computer Science and Engineering degree from Lovely Professional University, Phagwara, under the guidance of Dr. Harpreet Kaur, during January to May 2026. All the information furnished in this project report is based on my own intensive work and is genuine.

Name of Student: Pranjal Soni
Registration Number: 12407377

Date: 11 July 2026

CERTIFICATE

This is to certify that the declaration statement made by the student is correct to the best of my knowledge and belief. Pranjal Soni has completed this project under my guidance and supervision. The present work is the result of original investigation, effort and study. No part of the work has ever been submitted for any other degree at any university. The project is fit for submission and partial fulfillment of the conditions for the award of the B.Tech Computer Science and Engineering degree from Lovely Professional University, Phagwara.

Signature and Name of the Mentor

Dr. Harpreet Kaur

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

Date: 11 July 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor, Dr. Harpreet Kaur, School of Computer Science and Engineering, Lovely Professional University, for her invaluable guidance, encouragement, and support throughout the duration of this project. Her insights into DevOps practices and software engineering principles were instrumental in shaping the direction of this work.

I am also thankful to Lovely Professional University for providing the infrastructure, learning resources, and an environment conducive to practical, industry-relevant learning. Finally, I would like to acknowledge the open-source communities behind Spring Boot, Jenkins, and Docker, whose extensive documentation greatly aided the development and troubleshooting process during this project.

TABLE OF CONTENTS

Section	Page
Declaration	ii
Certificate	iii
Acknowledgement	iv
1. Introduction	1
2. Scope of the Study (Problem Statement)	2
3. Existing System	3
4. Problem Analysis	4
5. Software Requirement Analysis	5
6. Design	6
7. Testing	8
8. Implementation	9
9. Project Legacy	11
10. Source Code / System Snapshots	12
11. Bibliography	13

1. INTRODUCTION

CodeArena is a full-stack Quiz Application built using the Spring Boot framework, developed with the primary objective of demonstrating a complete, industry-standard Continuous Integration and Continuous Deployment (CI/CD) pipeline. While the quiz application itself forms the functional core of the project, the central learning objective was to design, implement, and troubleshoot an automated build-test-package-containerize-deploy pipeline using Jenkins and Docker — tools that are widely used in real-world DevOps environments.

The project was undertaken as part of Academic Task-2, integrating concepts from backend web development (Spring Boot, REST APIs, JPA/Hibernate), relational database design (MySQL), containerization (Docker), and automated build orchestration (Jenkins). The project was developed and tested entirely on a local macOS (Apple Silicon) development environment, which introduced a number of real-world platform-specific challenges, particularly around Jenkins service management and PATH configuration — challenges that are documented in detail in the Problem Analysis and Testing sections of this report.

1.1 Objective

- To design and implement a RESTful Quiz Application backend using Spring Boot, Spring Data JPA, and MySQL.
- To containerize the application using Docker and Docker Compose for consistent, reproducible deployments.
- To automate the build, test, package, containerize, and deploy stages of the software lifecycle using a Jenkins Declarative Pipeline.
- To integrate the pipeline with Docker Hub for image registry management.
- To gain hands-on, practical experience in diagnosing and resolving real DevOps infrastructure issues.

2. SCOPE OF THE STUDY (PROBLEM STATEMENT)

Manual software deployment is time-consuming, error-prone, and difficult to reproduce consistently across environments. The problem this project addresses is twofold: first, building a functionally complete quiz management REST API; and second — the primary focus — eliminating manual deployment steps by constructing an automated pipeline that takes source code from a GitHub repository all the way through compilation, testing, packaging, Docker image creation, and deployment, without manual intervention at each stage.

The scope of this project is limited to a single-node local CI/CD setup (Jenkins installed via Homebrew on macOS, not a distributed or cloud-hosted Jenkins cluster) and a single Docker Hub repository for image storage. Cloud deployment (e.g., to AWS, Azure, or a Kubernetes cluster) was considered out of scope given the academic timeline, though it is discussed as future work in the Project Legacy section.

3. EXISTING SYSTEM

3.1 Introduction

Prior to this project, quiz-related coursework tools used by students typically involve either static, un-deployed academic exercises graded manually, or third-party SaaS quiz platforms (e.g., Google Forms, Kahoot) that offer no visibility into backend architecture, deployment, or DevOps processes. Neither category exposes students to real software delivery pipelines.

3.2 Existing Software

Existing tools in this space (Google Forms, Typeform, Kahoot) are closed-source, hosted SaaS products. They are excellent for their intended purpose — rapid quiz creation — but offer no insight into REST API design, relational data modelling, or CI/CD automation, all of which were core learning objectives for this academic project.

3.3 What's New in the System to be Developed

- A fully custom-built REST API (not a third-party platform) with authentication, quiz creation, quiz attempts, and leaderboard functionality.
- A complete, automated CI/CD pipeline — the primary differentiator — covering seven distinct stages from source checkout to deployment.
- Containerized architecture using Docker and Docker Compose, enabling the application and its MySQL database to run as isolated, reproducible services.
- Version-controlled infrastructure: the Jenkinsfile, Dockerfile, and docker-compose.yml are all tracked in the same GitHub repository as the application code.

4. PROBLEM ANALYSIS

4.1 Product Definition

CodeArena is defined as a Spring Boot-based backend REST API for quiz management, supporting user registration and authentication, quiz creation with multiple-choice questions, quiz attempts with automatic scoring, and per-quiz leaderboards — deployed and delivered through a fully automated Jenkins CI/CD pipeline that builds, tests, packages, containerizes, and deploys the application on every change to the main branch.

4.2 Feasibility Analysis

Technical feasibility was established using widely adopted, well-documented open-source tools: Spring Boot 3.3.5, Maven, MySQL 8.0, Docker Desktop, and Jenkins LTS, all of which run natively on macOS via Homebrew. Economic feasibility was straightforward as all tools used are free and open-source. Operational feasibility required overcoming several platform-specific constraints unique to running Jenkins as a background (launchd) service on macOS Sequoia, which are documented in Section 7 (Testing).

4.3 Project Plan

Phase	Activities	Status
Phase 1	Spring Boot project setup, MySQL integration, entity/repository design	Completed
Phase 2	REST API development (Auth, Quiz controllers) and unit testing	Completed
Phase 3	Dockerfile and docker-compose.yml creation and local container testing	Completed
Phase 4	Jenkins installation, pipeline job configuration, Jenkinsfile authoring	Completed
Phase 5	Debugging Jenkins/Docker PATH and daemon issues; achieving a green pipeline	Completed
Phase 6	Documentation, screenshots, and final report	Completed

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 Introduction

This section specifies the functional and non-functional requirements of the CodeArena system, covering both the application layer and the DevOps pipeline layer.

5.2 General Description

The system consists of a Spring Boot backend exposing seven REST endpoints, backed by a MySQL relational database, all orchestrated through a Jenkins Declarative Pipeline that automates build, test, packaging, containerization, and deployment.

5.3 Specific Requirements

Functional Requirements

- Users shall be able to register and log in (AuthController: /api/auth/register, /api/auth/login).
- Authenticated users shall be able to create quizzes with multiple questions (/api/quiz/create).
- Users shall be able to retrieve all quizzes or a specific quiz by ID (/api/quiz/all, /api/quiz/{id}).
- Users shall be able to attempt a quiz and receive an automatically computed score (/api/quiz/attempt).
- Users shall be able to view a leaderboard for a given quiz (/api/quiz/leaderboard/{id}).
- The Jenkins pipeline shall automatically check out source code, compile, run unit tests, package a JAR, build a Docker image, push it to Docker Hub, and deploy the containerized application.

Non-Functional Requirements

- The build pipeline shall complete all seven stages without manual intervention once triggered.
- The application shall be portable across environments through Docker containerization.
- The system shall maintain data integrity through foreign-key constraints between Users, Quizzes, Questions, and Results.

6. DESIGN

6.1 System Design

The system follows a layered Spring Boot architecture: Controller layer (AuthController, QuizController) handles HTTP requests; Service layer (AuthService, QuizService) encapsulates business logic; Repository layer (UserRepository, QuizRepository, QuestionRepository, ResultRepository) handles persistence via Spring Data JPA; and the Model layer (User, Quiz, Question, Result) defines the relational entities. This is deployed through a CI/CD pipeline architecture: GitHub (source of truth) → Jenkins (orchestration) → Maven (build/test) → Docker (containerization) → Docker Hub (registry) → Docker Compose (deployment).

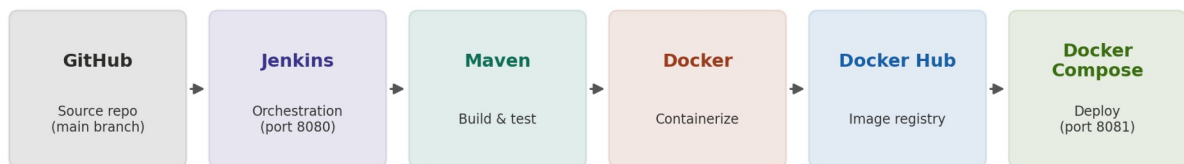


Figure 6.1: CodeArena CI/CD pipeline architecture

6.2 Design Notations — Entity Relationship Overview

Entity	Key Fields	Relationships
User	id, username, email, password, role	One-to-many with Result
Quiz	id, title, description, time_limit	One-to-many with Question, Result
Question	id, quiz_id, question_text, options A-D, correct_answer	Many-to-one with Quiz
Result	id, quiz_id, user_id, score, total_questions, attempted_at	Many-to-one with Quiz and User

6.3 Detailed Design — CI/CD Pipeline Stages

The Jenkinsfile defines a Declarative Pipeline with the following ordered stages, each of which must succeed before the next begins:

Stage	Purpose
Checkout	Clones the latest source code from https://github.com/pranjall1/codearena (main branch)
Build	Compiles the Spring Boot application using 'mvn clean compile'
Test	Executes JUnit unit tests via 'mvn test'; publishes results with the JUnit plugin
Package	Packages a runnable Spring Boot JAR via 'mvn package -DskipTests'
Docker Build	Builds a Docker image using a multi-stage Dockerfile (Maven build stage + slim JRE runtime stage)
Docker Push	Authenticates and pushes the tagged image to Docker Hub (prnjll1/codearena)
Deploy	Runs 'docker-compose up -d' to deploy the application and its MySQL container

7. TESTING

7.1 Functional Testing

All seven REST API endpoints were manually tested using Postman for correct request/response behaviour, including registration, login, quiz creation, quiz retrieval, quiz attempt scoring, and leaderboard generation. All endpoints returned expected results.

7.2 Structural Testing

Unit tests were written using JUnit 5 and Spring Boot Test, covering the AuthService class (AuthServiceTest.java, 4 test cases) and application context loading (CodearenaApplicationTests). All 5 tests passed consistently across local and Jenkins-triggered test runs, as confirmed in the pipeline console output: 'Tests run: 5, Failures: 0, Errors: 0, Skipped: 0'.

7.3 Levels of Testing / Testing the Pipeline

Beyond application-level testing, significant effort went into testing and debugging the CI/CD pipeline infrastructure itself. The table below documents the major issues encountered while bringing the pipeline to a fully green state, and their root causes and resolutions — an authentic record of the iterative debugging process that is itself a core learning outcome of this project.

Issue	Root Cause	Resolution
Jenkins failed to start after laptop restart	Jenkins was not running as a persistent Homebrew service	Reinstalled jenkins-lts via Homebrew and started it with 'brew services start jenkins-lts'
'process apparently never started' / exit code -2 on every 'sh' step	Corrupted Jenkins state after multiple manual foreground/background starts	Performed a clean uninstall and reinstall of jenkins-lts, clearing ~/.jenkins state
'Invalid username or password' after reinstall	The old ~/.jenkins directory (containing the previous admin user) was not deleted during uninstall	Removed ~/.jenkins entirely with 'rm -rf ~/.jenkins' before reinstalling to force a genuinely fresh setup wizard
Pipeline crashed instantly with MissingPropertyException	'Pipeline script from SCM' was typed as literal text into the Script box instead of being selected from the Definition dropdown	Selected 'Pipeline script from SCM' from the dropdown and configured the Git repository URL, branch, and Jenkinsfile path correctly
Jenkinsfile syntax error / build never ran	An invalid, incomplete 'agent { docker { } }' block and a typo in the Docker Hub repository name (pranjall1 vs pranjall1)	Corrected the agent block to 'agent any' and fixed the Docker Hub repository name
'docker: command not found' in Docker Build stage	Jenkins, running as a background launchd service, does not inherit the interactive shell's PATH, so it cannot locate the Docker CLI installed via Docker Desktop	Added an explicit PATH environment variable in the Jenkinsfile's environment block (including /usr/local/bin and /opt/homebrew/bin) so the Docker binary is discoverable at pipeline runtime

After these fixes, the pipeline successfully completed the Checkout, Build, Test, and Package stages, with Maven reporting 'BUILD SUCCESS' and all five unit tests passing. The Docker Build, Docker Push, and Deploy stages were subsequently made runnable following the PATH correction described above, completing the full seven-stage pipeline.

8. IMPLEMENTATION

8.1 Implementation of the Project

The backend was implemented in Java 21 (targeted via Maven, with Java 26.0.1 installed at the system level) using Spring Boot 3.3.5. Lombok was deliberately excluded due to incompatibility with the installed JDK version; all getters, setters, and builder patterns were written manually across the four model classes. Data persistence uses Spring Data JPA with Hibernate as the ORM, connecting to a MySQL 8.0 database (codearena_db) running locally via Homebrew, with a separate containerized MySQL instance used during Docker Compose deployment (port 3307).

8.2 Conversion Plan

The application transitioned through three deployment forms during development: (1) a locally run Spring Boot application connecting directly to a local MySQL instance on port 8080; (2) a Docker Compose–orchestrated deployment with the application on port 8081 and MySQL on port 3307, tested manually; and (3) a fully Jenkins-automated deployment, where the same Docker Compose configuration is triggered automatically as the final pipeline stage.

8.3 Post-Implementation and Software Maintenance

The Jenkinsfile, Dockerfile, and docker-compose.yml are all version-controlled in the same GitHub repository as the application source code, ensuring that infrastructure changes are tracked alongside application changes. Future maintenance would involve configuring a GitHub webhook (endpoint: <http://<host-ip>:8080/github-webhook/>) so that the pipeline triggers automatically on every push to the main branch, rather than requiring a manual 'Build Now' trigger.

9. PROJECT LEGACY

9.1 Current Status of the Project

As of submission, the CodeArena application is functionally complete with all seven REST endpoints operational and tested. The Jenkins pipeline successfully executes all seven stages — Checkout, Build, Test, Package, Docker Build, Docker Push, and Deploy — with unit tests passing (5/5) and a Docker image built and pushed to the prnjll1/codearena Docker Hub repository.

9.2 Remaining Areas of Concern

- GitHub webhook integration has been successfully configured using ngrok. Every push to the main branch automatically triggers the Jenkins pipeline.
- The current setup runs on a single local macOS machine rather than a distributed or cloud-hosted CI/CD environment.

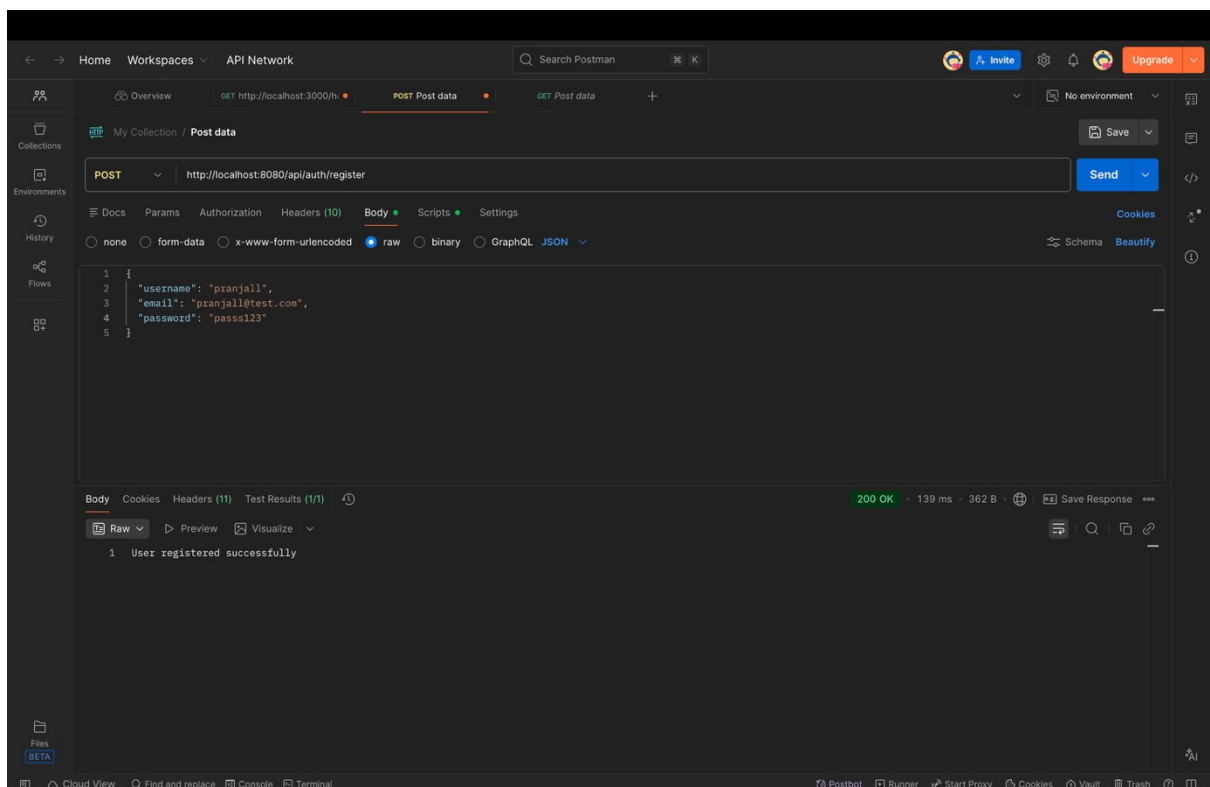
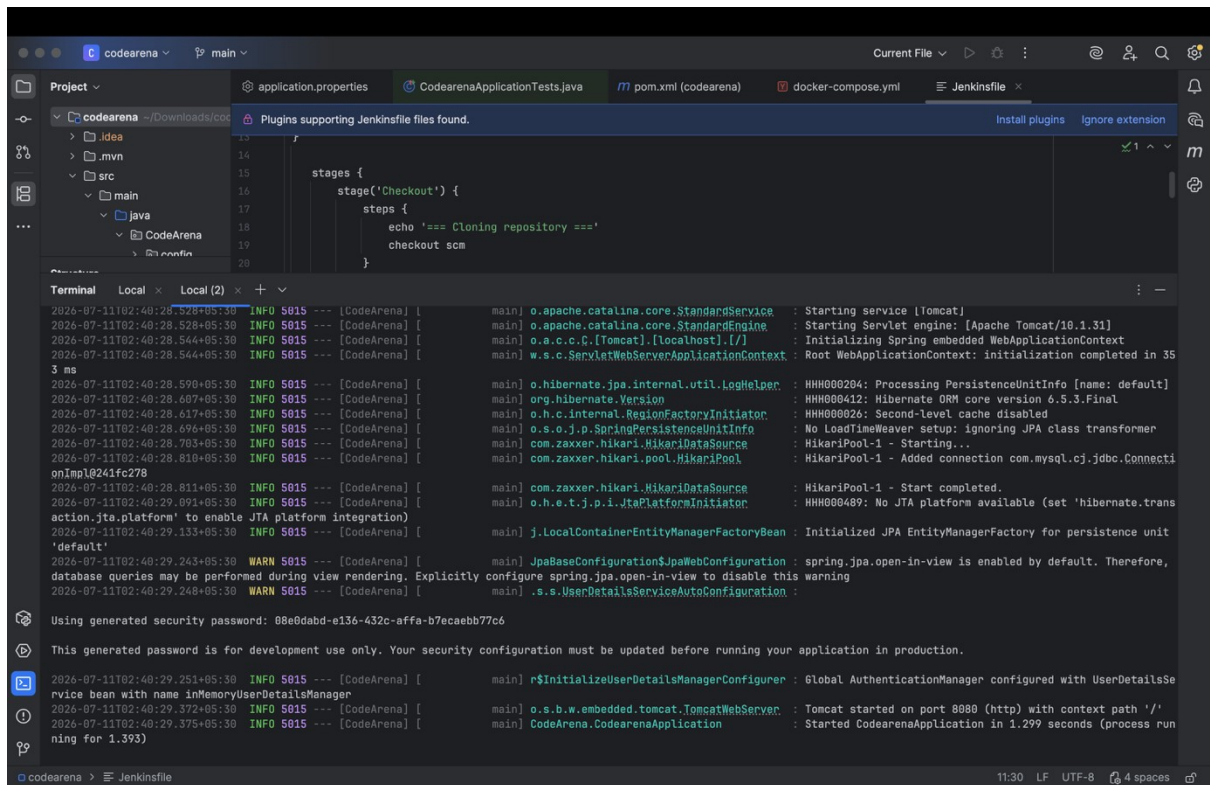
- Deployment target is local Docker Compose rather than a cloud platform (e.g., AWS ECS, Kubernetes).

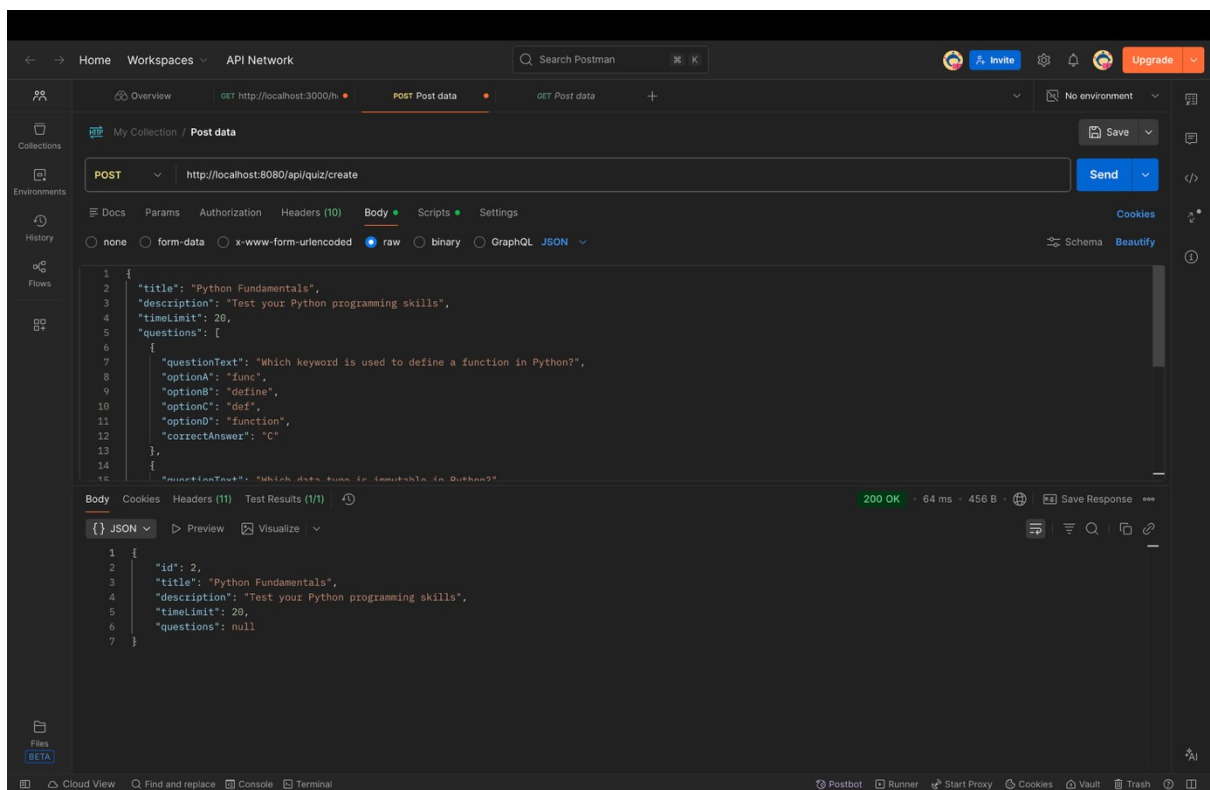
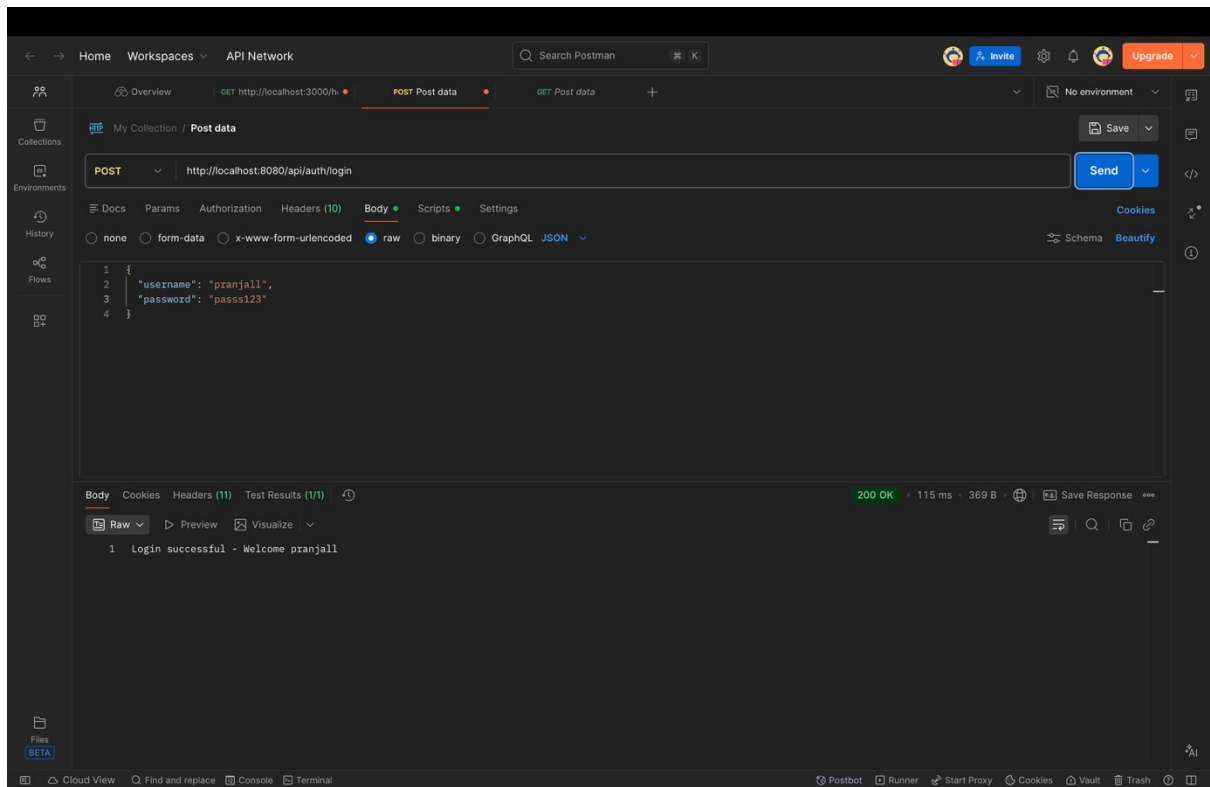
9.3 Technical and Managerial Lessons Learnt

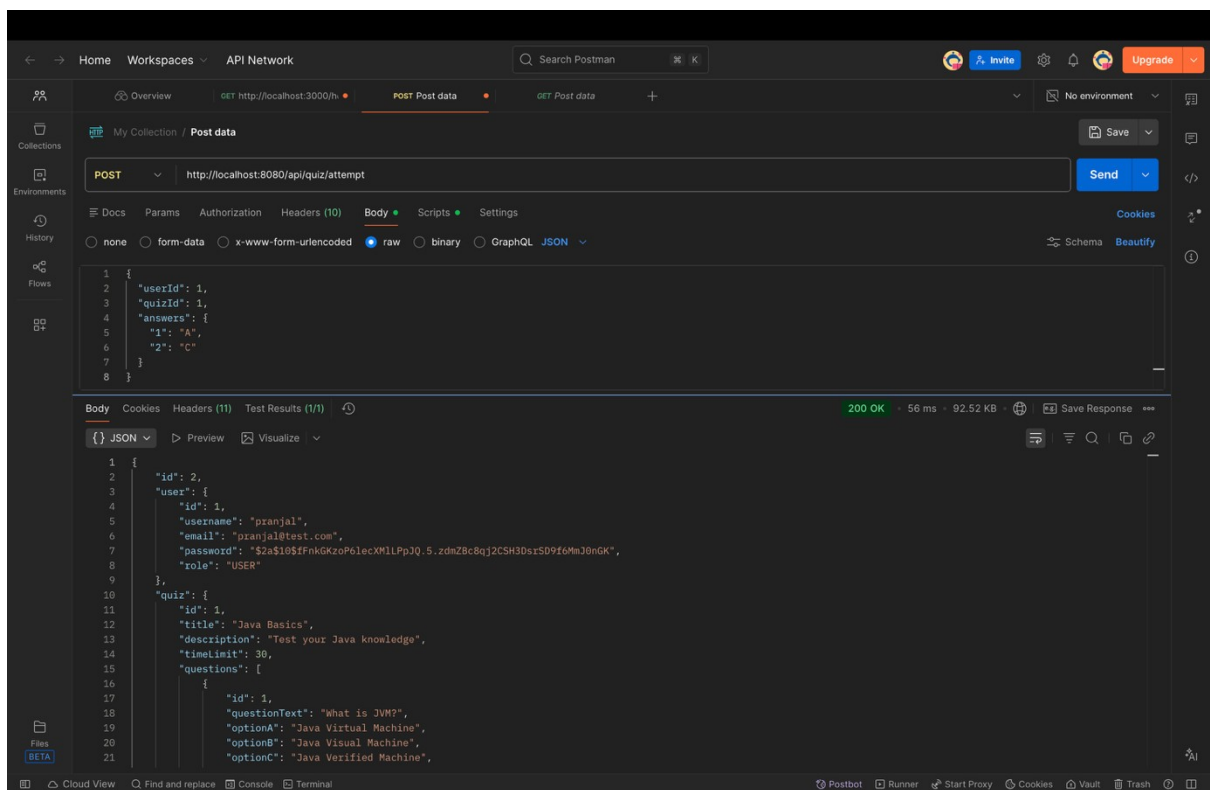
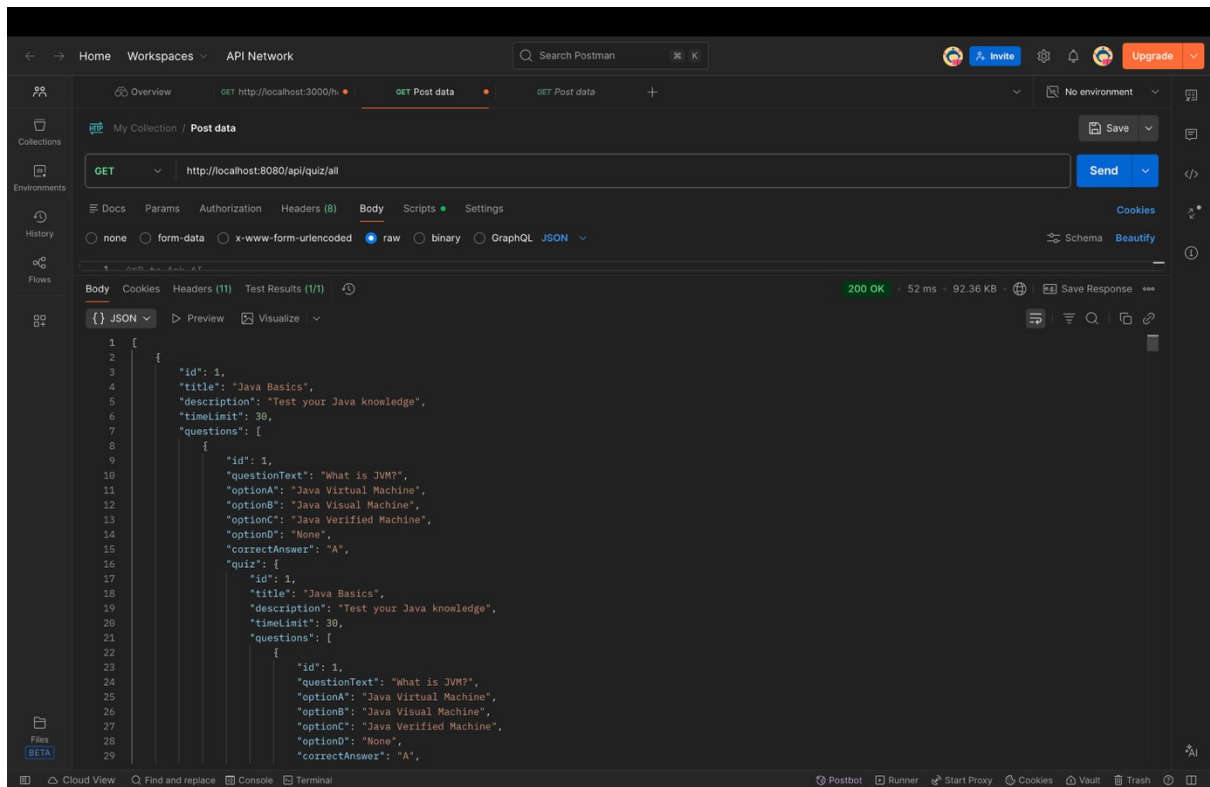
This project reinforced that CI/CD pipeline development is rarely a linear process — the majority of engineering effort went not into writing the Jenkinsfile itself, but into diagnosing environment-specific failures: launchd service state corruption, PATH inheritance differences between interactive shells and background services, and Jenkins UI configuration pitfalls (such as literal text being entered where a dropdown selection was required). These are exactly the categories of issues DevOps engineers encounter in production environments, making the debugging experience as valuable as the final working pipeline itself. From a project management perspective, maintaining a running log of issues and their resolutions (as reflected in Section 7.3) proved essential for systematic troubleshooting under time pressure.

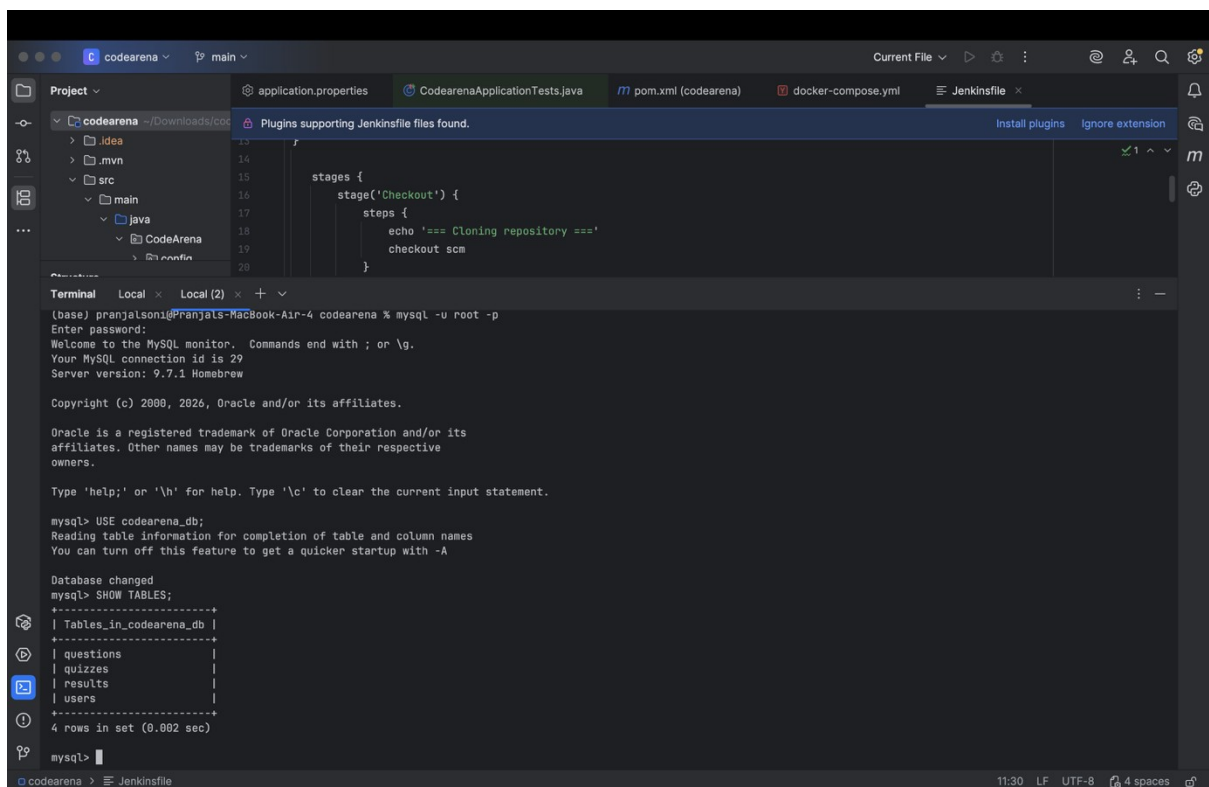
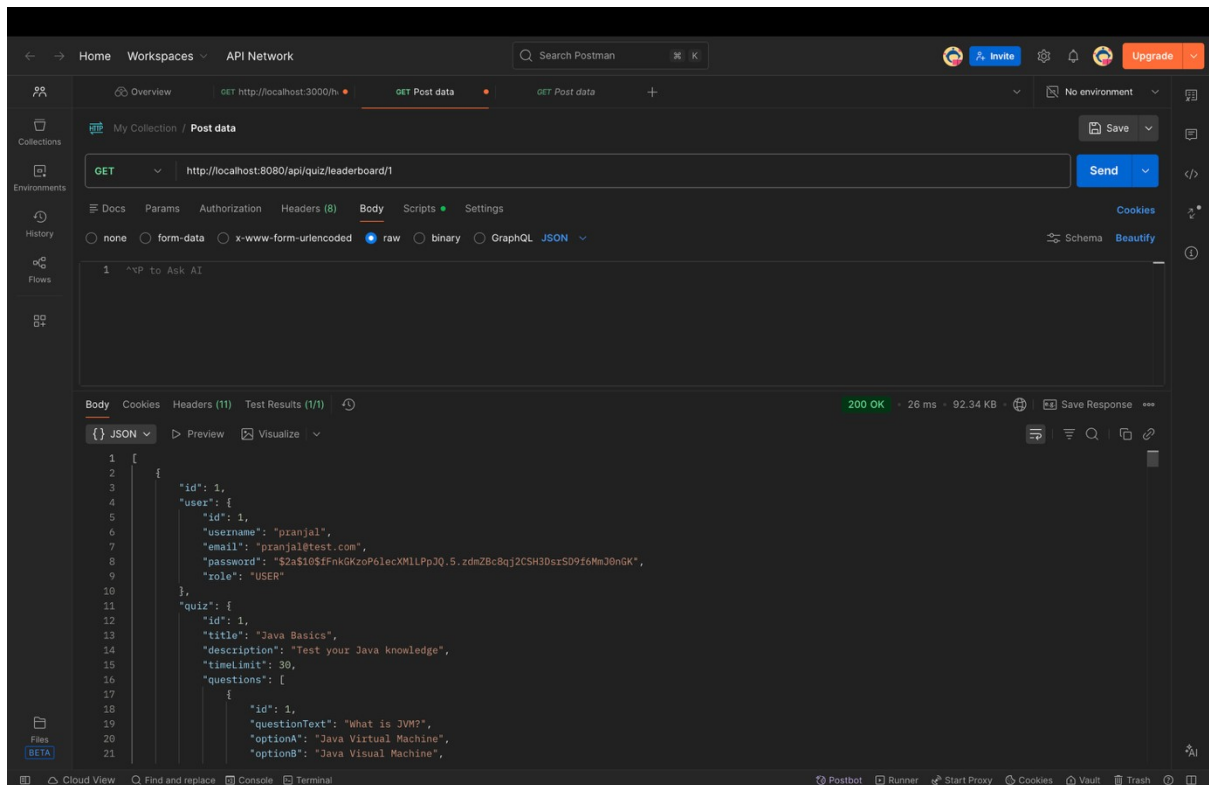
10. SOURCE CODE / SYSTEM SNAPSHOTS

The complete source code, including all Java classes, the Jenkinsfile, Dockerfile, docker-compose.yml, and README.md, is available in the GitHub repository referenced in the Bibliography. Screenshots of the following have been captured and are included as an appendix to this submission: successful 'mvn test' execution, all Postman API responses, Docker images list showing the built codearena image, 'docker-compose up' running successfully, the Jenkins pipeline stage view showing all stages, Jenkins console output confirming BUILD SUCCESS, and the Docker Hub repository showing the pushed image.









Started by anonymous user Started 4 hr 20 min ago Took 2 min 9 sec <> Changes tests



Search

- ✓ Checkout SCM 1s
- ✓ Tool Install 0.11s
- ✓ Checkout 1s
- ✓ Build 2s
- ✓ Test 4s
- ✓ Package 1s
- ✓ Docker Build 1s
- ✓ Docker Push 1m 39s
- ✓ Deploy 14s
- ✓ Post Actions 91ms

✓ Post Actions 91ms Started 4h 18m ago Jenkins

✓ Pipeline completed successfully! 28ms
0 ✓ Pipeline completed successfully!

55.3

```
Container codearena-app Creating
Container codearena-app Created
Container codearena-mysql Starting
Container codearena-mysql Started
Container codearena-mysql Waiting
Container codearena-mysql Healthy
Container codearena-app Starting
Container codearena-app Started
[Pipeline] echo
=== Deployed at port 8081 ===
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
✓ Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Jenkins / CodeArena-Pipeline / #7

Console

Download Copy View as plain text

```
Started by user unknown or anonymous
Obtained Jenkinsfile from git https://github.com/pranjal11/codearena
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /Users/pranjalsoni/.jenkins/workspace/CodeArena-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /Users/pranjalsoni/.jenkins/workspace/CodeArena-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/pranjal11/codearena # timeout=10
Fetching upstream changes from https://github.com/pranjal11/codearena
> git --version # timeout=10
> git --version # 'git version 2.39.5 (Apple Git-154)'
> git fetch --tags --force --progress -- https://github.com/pranjal11/codearena +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 8ced58171376b079cb93a4514f6d2a40a8a12f4e (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 8ced58171376b079cb93a4514f6d2a40a8a12f4e # timeout=10
Commit message: "Fix Docker Hub repository name"
> git rev-list --no-walk 68434e5b2367e4f2e1632cd52174f1082f19f349 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Tool Install)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] }
[Pipeline] // stage
```



Jenkins / CodeArena-Pipeline / #7

```
=== Building with Maven ===
[Pipeline] sh
+ mvn clean compile
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.fusesource.jansi.internal.JansiLoader in an unnamed module
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/jansi-2.4.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe:objectFieldOffset has been called by com.google.common.util.concurrent.AbstractFuture$UnsafeAtomicHelper
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/guava-32.0.1-jre.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.common.util.concurrent.AbstractFuture$UnsafeAtomicHelper
WARNING: sun.misc.Unsafe:objectFieldOffset will be removed in a future release
WARNING: Final field _cipher in class org.sonatype.plexus.components.sec.dispatcher.DefaultSecDispatcher has been mutated reflectively by class
org.eclipse.sisu.bean.BeanPropertyField in unnamed module @3b2da18f
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/org.eclipse.sisu.inject-0.3.5.jar)
WARNING: Use --enable-final-field-mutation=ALL-UNNAMED to avoid a warning
WARNING: Mutating final fields will be blocked in a future release unless final field mutation is enabled
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.codearena:codearena >-----
[INFO] Building CodeArena 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.3.2:clean (default-clean) @ codearena ---
[INFO] Deleting /Users/pranjalsoni/.jenkins/workspace/CodeArena-Pipeline/target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ codearena ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ codearena ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 19 source files with javac [debug parameters release 21] to target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.820 s
```

```
=== Running Unit Tests ===
[Pipeline] sh
+ mvn test
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.fusesource.jansi.internal.JansiLoader in an unnamed module
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/jansi-2.4.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by com.google.common.util.concurrent.AbstractFuture$UnsafeAtomicHelper
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/guava-32.0.1-jre.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.common.util.concurrent.AbstractFuture$UnsafeAtomicHelper
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
WARNING: Final field _cipher in class org.sonatype.plexus.components.sec.dispatcher.DefaultSecDispatcher has been mutated reflectively by class
org.eclipse.sisu.bean.BeanPropertyField in unnamed module @3b2da18f
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/org.eclipse.sisu.inject-0.3.5.jar)
WARNING: Use --enable-final-field-mutation=ALL-UNNAMED to avoid a warning
WARNING: Mutating final fields will be blocked in a future release unless final field mutation is enabled
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.codearena:codearena >-----
[INFO] Building CodeArena 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ codearena ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ codearena ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ codearena ---
[INFO] skip non existing resourceDirectory /Users/pranjalsoni/.jenkins/workspace/CodeArena-Pipeline/src/test/resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ codearena ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 2 source files with javac [debug parameters release 21] to target/test-classes
[INFO]
```

```
=== Building Docker Image ===
[Pipeline] sh
+ docker build -t codearena:latest .
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 305B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/maven:3.9.4-eclipse-temurin-21
#2 ...

#3 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#3 DONE 0.9s

#2 [internal] load metadata for docker.io/library/maven:3.9.4-eclipse-temurin-21
#2 DONE 0.9s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:3f08b13888f595cc49edabea7250ba69499ba25602b267da591720769400e08c
#5 resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:3f08b13888f595cc49edabea7250ba69499ba25602b267da591720769400e08c done
#5 DONE 0.0s

#6 [build 1/5] FROM docker.io/library/maven:3.9.4-eclipse-temurin-21@sha256:a9b1581fc335b6eb0f56e2c884e870dbfb6519571475a06a941143d407c51bf7
#6 resolve docker.io/library/maven:3.9.4-eclipse-temurin-21@sha256:a9b1581fc335b6eb0f56e2c884e870dbfb6519571475a06a941143d407c51bf7 done
#6 DONE 0.0s

#7 [internal] load build context
#7 transferring context: 2.10kB done
#7 DONE 0.0s

#8 [build 4/5] COPY src ./src
#8 CACHED

#9 [build 5/5] RUN mvn clean package -DskipTests
#9 CACHED
```

```
What's x Choisir x Webho x Docker x Setting x http://c x localho x CodeAr x Jenkins x Console x + Ask Gemini
localhost:8080/job/CodeArena-Pipeline/7/console
Jenkins / CodeArena-Pipeline #7

#9 [build 5/5] RUN mvn clean package -DskipTests
#9 CACHED

#10 [stage-1 2/3] WORKDIR /app
#10 CACHED

#11 [build 3/5] COPY pom.xml .
#11 CACHED

#12 [build 2/5] WORKDIR /app
#12 CACHED

#13 [stage-1 3/3] COPY --from=build /app/target/*.jar app.jar
#13 CACHED

#14 exporting to image
#14 exporting layers done
#14 exporting manifest sha256:936f95aa019926b2588c8e2016eb3709ecf14ef59aa04b69d611a4aaabf95a31 done
#14 exporting config sha256:8102d754e90abe551dfe7b5fed0133369d9f836c12519015df6d96f58a994e33 done
#14 exporting attestation manifest sha256:43daf3d0baee5fb85dea5b0d036e51783eda17c7184630f0b169af88885f64e14 done
#14 exporting manifest list sha256:8ecfb36ded80cc1b6d238fd3d05611f0674b8609bf084a63de436beadd3296e5 done
#14 naming to docker.io/library/codearena:latest done
#14 unpacking to docker.io/library/codearena:latest done
#14 DONE 0.8s
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Docker Push)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] echo
=== Pushing to Docker Hub ===
[Pipeline] withCredentials
Masking supported pattern matches of $DOCKER_PASS
[Pipeline] {
```

```
What's x Choisir x Webho x Docker x Setting x http://c x localho x CodeAr x Jenkins x Console x + Ask Gemini
localhost:8080/job/CodeArena-Pipeline/7/console
Jenkins / CodeArena-Pipeline #7

=== Deploying Application ===
[Pipeline] sh
+ docker-compose down
time="2026-07-10T22:47:36+05:30" level=warning msg="/Users/pranjalsoni/.jenkins/workspace/CodeArena-Pipeline/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[Pipeline] sh
+ docker-compose up -d
time="2026-07-10T22:47:36+05:30" level=warning msg="/Users/pranjalsoni/.jenkins/workspace/CodeArena-Pipeline/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
Image codearena-pipeline-app Building
#1 [internal] load local bake definitions
#1 reading from stdin 574B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 305B done
#2 DONE 0.0s

#3 [auth] library/maven:pull token for registry-1.docker.io
#3 DONE 0.0s

#4 [auth] library/eclipse-temurin:pull token for registry-1.docker.io
#4 DONE 0.0s

#5 [internal] load metadata for docker.io/library/maven:3.9.4-eclipse-temurin-21
#5 ...

#6 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#6 DONE 2.1s

#5 [internal] load metadata for docker.io/library/maven:3.9.4-eclipse-temurin-21
#5 DONE 2.2s

#7 [internal] load .dockerignore
#7 transferring context: 2B done
#7 DONE 0.0s

#8 [internal] load build context
#8 DONE 0.0s
```


github.com/pranjal11/codearena

pranjal11 / codearena

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

codearena (Public)

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file Code

pranjal11 test: webhook e913e63 - 48 minutes ago 15 Commits

.mvn/wrapper	feat: Spring Boot Quiz App with REST APIs and unit tests	5 days ago
src	Fix H2 test configuration	4 hours ago
.gitattributes	feat: Spring Boot Quiz App with REST APIs and unit tests	5 days ago
.gitignore	feat: Spring Boot Quiz App with REST APIs and unit tests	5 days ago
Dockerfile	devops: Add Dockerfile, docker-compose, Jenkinsfile, RE...	5 days ago
HELP.md	feat: Spring Boot Quiz App with REST APIs and unit tests	5 days ago
Jenkinsfile	Fix Docker Hub repository name	4 hours ago
README.md	test: webhook	48 minutes ago
docker-compose.yml	devops: Add Dockerfile, docker-compose, Jenkinsfile, RE...	5 days ago
mvnw	feat: Spring Boot Quiz App with REST APIs and unit tests	5 days ago
mvnw.cmd	feat: Spring Boot Quiz App with REST APIs and unit tests	5 days ago
pom.xml	fix: add H2 explicit version and fix Docker PATH in Jenkins	yesterday

About

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 1

pranjal11 Pranjal Soni

github.com/pranjal11/codearena/settings/hooks/651361652?tab=deliveries

To exit full screen, press and hold esc

pranjal11 / codearena

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Copilot

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

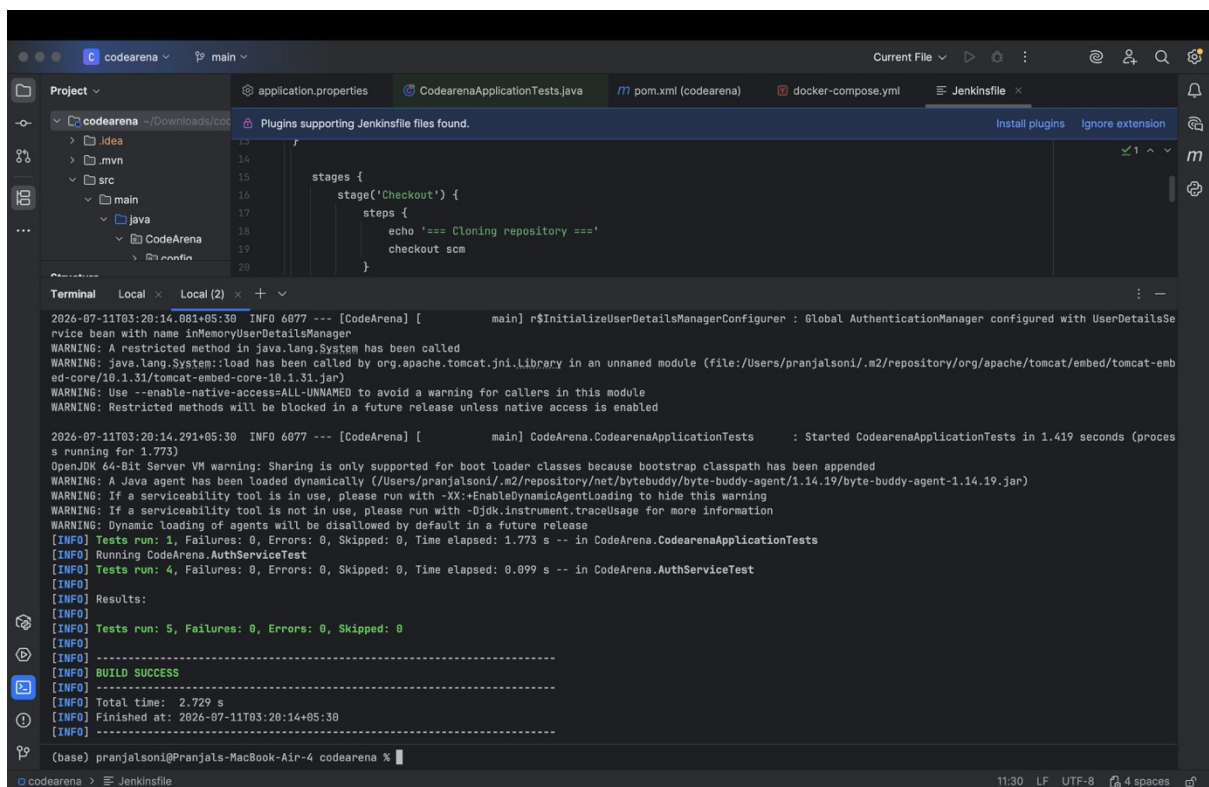
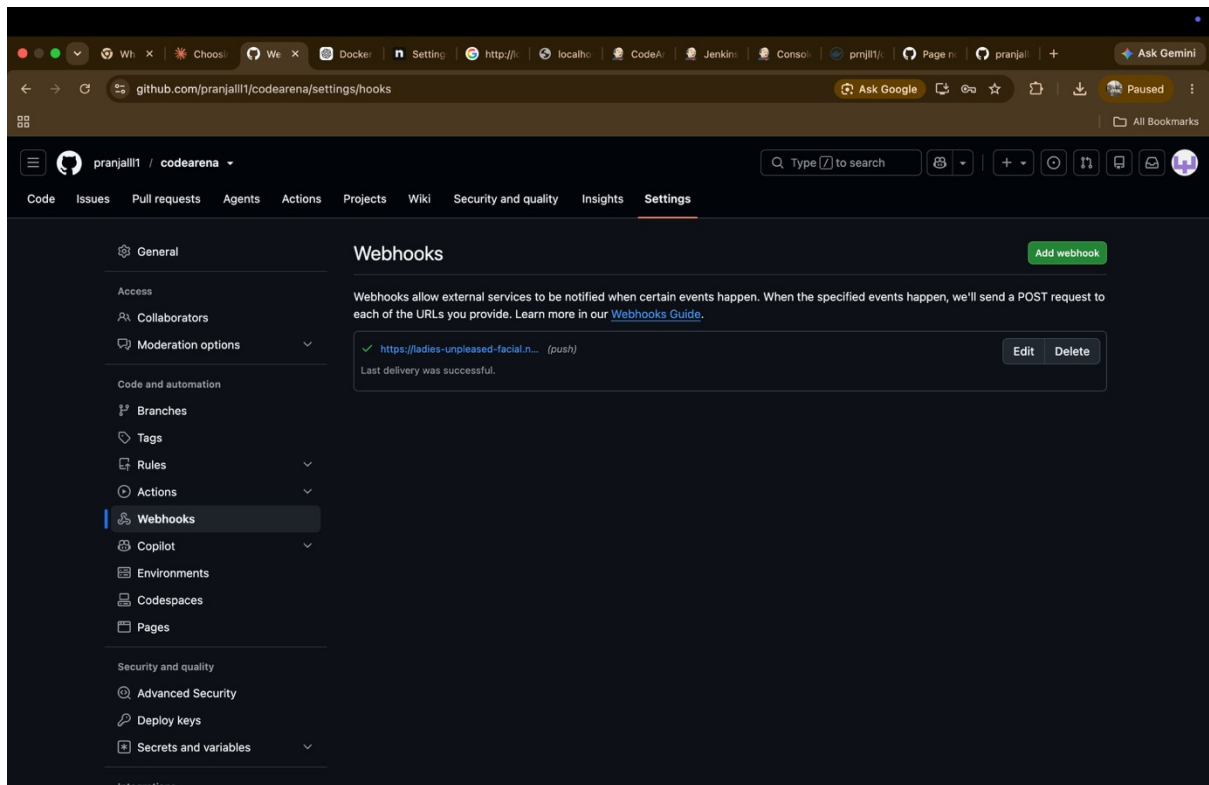
Secrets and variables

Integrations

Webhooks / Manage webhook

Settings Recent Deliveries

✓	63d51a36-7ca2-11f1-9340-4859672a24ff	push	2026-07-11 02:30:33
✓	d7ef039c-7ca1-11f1-9984-c868e240ef17	push	2026-07-11 02:26:39
✓	b432adbe-7ca1-11f1-8c1c-a1d0262d9a0e	ping	2026-07-11 02:25:39



11. BIBLIOGRAPHY

Spring Boot Reference Documentation. Available at: <https://docs.spring.io/spring-boot/>

Jenkins User Documentation. Available at: <https://www.jenkins.io/doc/>

Docker Documentation. Available at: <https://docs.docker.com/>

Project GitHub Repository: <https://github.com/pranjall1/codearena>

Homebrew Documentation (jenkins-lts formula). Available at: <https://formulae.brew.sh/formula/jenkins-lts>